University of Bologna

Electrical engineering degree, curriculum: Electronics for intelligent systems, big-data and internet of things

---

Analysis and circuit implementation of Fourier Transform based on Phase in Memory Computing technology

Supervisor:

Candidate:

Luca Lizzi

Accademic year 2025/2026

Contents

1 Introduction **1**
1.1 State of the art 1

2 Algortims **2**
2.1 DFT and then FFT 2
2.2 Sliding DFT 3
2.3 R-SDFT and M-SDFT 4
2.3.1 R-SDFT 4
2.3.2 M-SDFT 6
2.4 DC considerations 7
2.4.1 Resonator-based SDFT considering the DC component 8
2.4.2 Modulator-based SDFT considering the DC component 8

3 Results **9**
3.1 Ideal (float64) 10
3.1.1 Magnitude error vs frequency 10
3.1.2 Complex error (phase) vs frequency 11
3.1.3 Heatmaps 11
3.2 Quantized versions 11
3.2.1 Magnitude error vs frequency 13
3.2.2 Complex error (phase) vs frequency 14
3.2.3 Heatmaps 14
3.3 Circuital error 14
3.3.1 Magnitude error vs frequency 16
3.3.2 Complex error (phase) vs frequency 16
3.3.3 Heatmaps 18

4 Electric circuit idea **19**
4.1 Simulations 19

5 Conclusions **21**

Chapter 1

Introduction

Qui scriverai l'introduzione della tua tesi...

1.1 State of the art

Qui descriverai la teoria e lo stato dell'arte...

Chapter 2

Algortims

2.1 DFT and then FFT

The Discrete Fourier Transform (DFT) maps a finite sequence of N_0 samples, $x[n]$, to its frequency-domain representation, X_k. Using the forward-transform convention adopted here, the DFT and its inverse are

$$X_k = \frac{1}{N_0} \sum_{n=0}^{N_0-1} x[n] W_{N_0}^{kn},$$
$$x[n] = \sum_{k=0}^{N_0-1} X_k W_{N_0}^{-kn},$$

where $n, k \in \{0, 1, \ldots, N_0 - 1\}$ and

$$W_{N_0} = e^{-j2\pi/N_0}$$

is the primitive N_0-th root of unity. Its powers, $W_{N_0}^{kn}$, are called twiddle factors.

A direct evaluation of the DFT requires N_0^2 complex multiplications and $N_0(N_0 - 1)$ complex additions. Counting a general complex multiplication as four real multiplications and two real additions, and a complex addition as two real additions, this corresponds to $4N_0^2$ real multiplications and $4N_0^2 - 2N_0$ real additions. This count assumes that no operations are eliminated for special twiddle factors. Therefore, the direct DFT has a computational complexity of $O(N_0^2)$.

The Fast Fourier Transform (FFT) computes the same DFT values without approximation while reducing the number of operations. The two common radix-2 forms are decimation in time (DIT) and decimation in frequency (DIF). The derivation below uses the DIT form and assumes that N_0 is a power of two.

In the first DIT decomposition, the input sequence is separated into its even- and odd-indexed samples:

$$e[n] = x[2n], \qquad o[n] = x[2n + 1], \qquad 0 \le n < \frac{N_0}{2}.$$

Let E_k and O_k denote the $N_0/2$-point DFTs of $e[n]$ and $o[n]$, respectively. For $0 \le k <$

$N_0/2$, splitting the DFT sum into its even- and odd-indexed terms gives

$$\begin{aligned} X_k &= \sum_{n=0}^{N_0/2-1} x[2n] W_{N_0}^{2kn} + \sum_{n=0}^{N_0/2-1} x[2n+1] W_{N_0}^{k(2n+1)} \\ &= \sum_{n=0}^{N_0/2-1} e[n] W_{N_0/2}^{kn} + W_{N_0}^{k} \sum_{n=0}^{N_0/2-1} o[n] W_{N_0/2}^{kn} \\ &= E_k + W_{N_0}^{k} O_k. \end{aligned}$$

The other half of the output follows from $W_{N_0}^{N_0/2} = -1$:

$$X_{k+N_0/2} = E_k - W_{N_0}^{k} O_k, \qquad 0 \le k < \frac{N_0}{2}.$$

Together, these two equations define a butterfly operation showed in Fig.2.1: for inputs E_k and O_k, the outputs are $E_k + W_{N_0}^{k} O_k$ and $E_k - W_{N_0}^{k} O_k$.

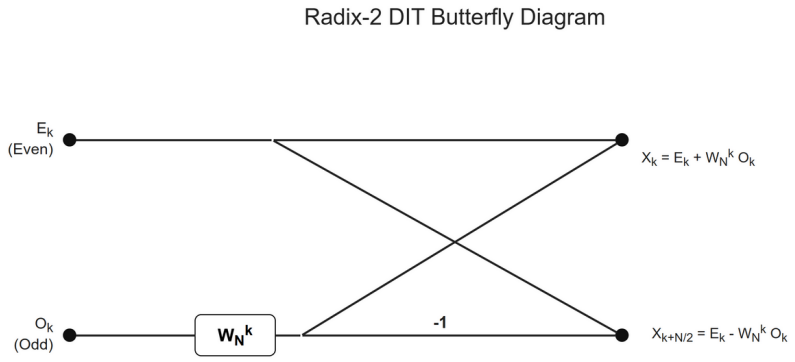


Figure 2.1: Radix-2 DIT butterfly diagram

For $N_0 = 2^m$, a radix-2 FFT has $m = \log_2 N_0$ stages, each containing $N_0/2$ butterflies. Counting one twiddle-factor multiplication, including trivial factors, and two complex additions per butterfly, the algorithm requires $(N_0/2)\log_2 N_0$ twiddle-factor multiplications and $N_0 \log_2 N_0$ complex additions. Its computational complexity is therefore $O(N_0 \log_2 N_0)$.

2.2 Sliding DFT

The FFT algorithm requires the entire signal to calculate the spectrum. However, some applications (such as tuning vibration neutralizers) require a faster algorithm that operates in real time. The article [3] presents two recursive algorithms, both referred to as the sliding DFT (SDFT). The core idea is to compute the initial spectrum using the first N_0 samples and then update it recursively whenever a new sample arrives, dropping the oldest one. In Fig.2.2 is showed an example of a sliding window.

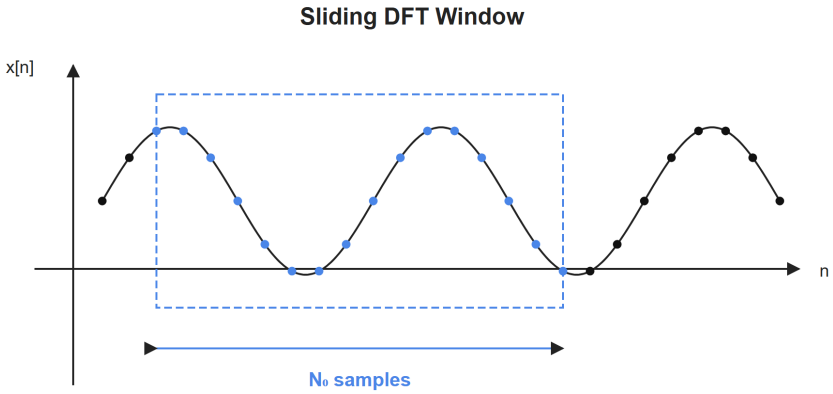


Figure 2.2: Enter Caption

2.3 R-SDFT and M-SDFT

In the article [3] are studied 2 algorithms: resonator and modulator used in branches included in a bigger recursive structure showed in the figure Fig. 2.3.

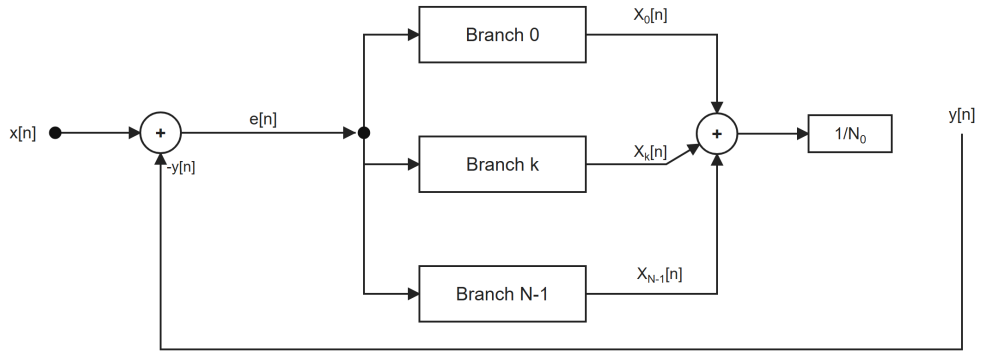


Figure 2.3: Resonator-based sliding DFT branch

2.3.1 R-SDFT

In the Fig.2.4 is showed the resonator branch. In the following the calculations to find the relation of this algorithm.

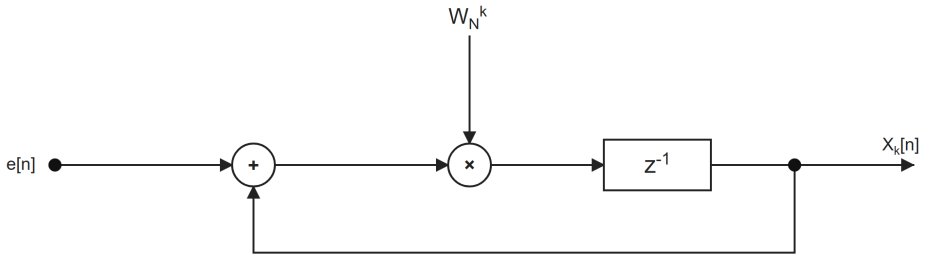


Figure 2.4: Resonator-based sliding DFT branch

$$\begin{aligned} t[n] &= r[n] \cdot W_{N_0}^k = (e[n] + X_k[n]) \cdot W_{N_0}^k \\ X_k[n] &= t[n-1] = (e[n-1] + X_k[n-1]) \cdot W_{N_0}^k \end{aligned} \xrightarrow{\text{Z-transform}} \begin{aligned} X_k(z) &= (E(z)z^{-1} + X_k(z)z^{-1}) \cdot W_N^k \\ X_k(z) &= E(z) \cdot \frac{W_N^k z^{-1}}{1 - W_N^k z^{-1}} \end{aligned}$$

This structure has a pole in W_N^k for this reason the structure is more affected by the quantization error with respect to the modulator-SDFT.
In the Listing 2.1 is showed the resonator-SDFT.

```
def rsdft(input_data:None, N:10, sample_rate:int=1, quant_en:bool=True, num_bits:int
    =8, max_amplitude:float=1.0,
          r_ideal:float=1.0, r_quant:float=1.0, hw_err:bool=False):

    input_data = input_data[:N]
    input_size = input_data.size

    k_vec = np.arange(N)
    if hw_err:
        rng = np.random.default_rng()
        k_vec = k_vec + rng.uniform(low=LOW_ERR, high=HIGH_ERR, size=N)
    arg = 2 * np.pi * k_vec / N

    # check of the constant
    v = [1.0]
    if SEARCH_R and ((quant_en and r_quant in v) or (not quant_en and r_ideal in v)):
        ok = False
        while(ok==False):
            if quant_en:
                twiddles_quant = quantizer(r_quant * np.exp(-1j * arg), num_bits=
                    num_bits)
                if np.all(np.abs(twiddles_quant) <= 1.0): ok = True
                else: r_quant -= 0.001
            else:
                twiddles_ideal = r_ideal * np.exp(-1j * arg)
                if np.all(np.abs(twiddles_ideal) <= 1.0): ok = True
                else: r_ideal -= 0.001
    else:
        if quant_en:    twiddles_quant = quantizer(r_quant * np.exp(-1j * arg),
            num_bits=num_bits)
        else:           twiddles_ideal = r_ideal * np.exp(-1j * arg)

    # variables
    out = np.zeros(input_size, dtype=np.float64)
    Z = np.zeros(N, dtype=np.complex128)
```

```
    prev_out = 0.0
    err = 0.0

    for index in range(input_size):
        if quant_en:
            err = quantizer(input_data[index] - prev_out, num_bits=num_bits)
            Z = quantizer(quantizer((err + Z), num_bits=num_bits) * twiddles_quant,
                num_bits=num_bits)
            prev_out = quantizer(np.sum(np.real(Z)) / N, num_bits=num_bits)
        else:
            err = input_data[index] - prev_out
            Z = (err + Z) * twiddles_ideal
            prev_out = np.sum(np.real(Z)) / N

        out[index] = prev_out

    return Z, r_ideal, r_quant
```

Listing 2.1: Resonator-based sliding DFT Python code

2.3.2 M-SDFT

In the Fig.2.5 is showed the modulator branch. In the following the calculations to find the relation of this algorithm. This algortihm can be divided in 3 parts: modulation, integration and demodulation. The modulation and the demodulation parts are defined by the multiplication of $W_{N_0}^{kn}$ and $W_{N_0}^{-kn}$, instead the integrator has the following transfer function.

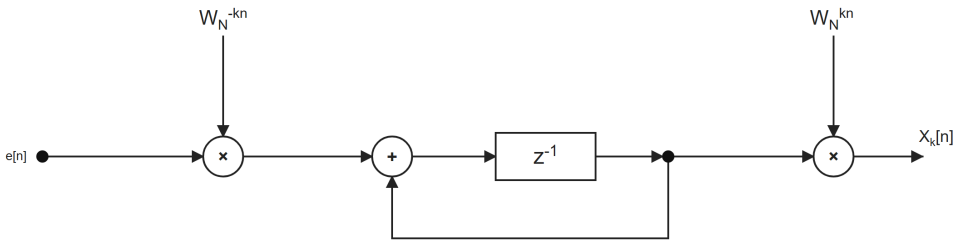


Figure 2.5: Modulator-based sliding DFT branch

$$\begin{aligned} u[n] &= e[n] \cdot W_{N_0}^{-kn} \\ v[n] &= u[n] + v[n-1] \end{aligned} \xrightarrow{\text{Z-transform}} \begin{aligned} V(z) &= U(z) + V(z)z^{-1} \\ \frac{V(z)}{U(z)} &= \frac{1}{1-z^{-1}} \end{aligned}$$

The pole in $|z| = 1$ ensures more stability than the resonator one from the fact is less affected by the quantization algorithm.
In the Listing 2.2 is shoed the modulator-based SDFT Python code.

```python
def msdft(input_data:None, N:10, sample_rate:int=1, quant_en:bool=True, num_bits:int=8, max_amplitude:float=1.0,
          r_ideal:float=1.0, r_quant:float=1.0, hw_err:bool=False):

    input_data = input_data[:N]
    input_size = input_data.size

    k_vec = np.arange(N)
    if hw_err:
        rng = np.random.default_rng()
        k_vec = k_vec + rng.uniform(low=LOW_ERR, high=HIGH_ERR, size=N)
    arg = 2 * np.pi * k_vec / N

    # check of the constant
    v = [1.0]
    if SEARCH_R and ((quant_en and r_quant in v) or (not quant_en and r_ideal in v)):
        ok = False
        while(ok==False):
            if quant_en:
                for n in range(input_size):
                    twiddles_quant = quantizer(r_quant * np.exp(-1j * arg * n), num_bits=num_bits)
                    if not np.all(np.abs(twiddles_quant) <= 1.0):
                        r_quant -= 0.001
                        break
                    else:   ok = True
            else:
                for n in range(input_size):
                    twiddles_ideal = r_ideal * np.exp(-1j * arg * n)
                    if not np.all(np.abs(twiddles_ideal) <= 1.0):
                        r_ideal -= 0.001
                        break
                    else:   ok = True

    # variables
    out = np.zeros(input_size, dtype=np.float64)
    Z = np.zeros(N, dtype=np.complex128)
    prev_out = 0.0
    err = 0.0

    for index in range(input_size):
        if quant_en:
            err = quantizer(input_data[index] - prev_out, num_bits=num_bits)
            twiddle = quantizer(r_quant * np.exp(-1j * arg * index), num_bits=num_bits)
            prev_out = quantizer(np.sum(np.real(Z * np.conjugate(twiddle))) / N, num_bits=num_bits)
            Z = quantizer(quantizer(r_quant * Z, num_bits=num_bits) +
                    quantizer(err * twiddle, num_bits=num_bits), num_bits=num_bits)
        else:
            err = input_data[index] - prev_out
            twiddle = r_ideal * np.exp(-1j * arg * index)
            prev_out = np.sum(np.real(Z * np.conjugate(twiddle))) / N
            Z = r_ideal * Z + err * twiddle

        out[index] = prev_out

    return Z, r_ideal, r_quant
```

Listing 2.2: Modulator-based sliding DFT Python code

2.4 DC considerations

Working on the progect has been discovered a way to reduce energy consumption in the optic of an hardware implementation and is applicable to the general structure of Fig.2.3. Consist on considering only the DC component during the construction of the input signal.

From the definition of the spectrum components $X_0 = \sum_{n=0}^{N_0-1} x[n]$ so the reconstructed output is: $y[n] = x[n]$. This means there is no need of considering all other elements of the spectrum because the error is memorized from the previous data sample.

2.4.1 Resonator-based SDFT considering the DC component

In Listing 2.3 is showed the algorithm of the R-SDFT considering the DC component in the signal reconstruction. All the rest of the function is equal to the R-SDFT function.

```
for index in range(input_size):
    if quant_en:
        err = quantizer(input_data[index] - prev_out, num_bits=num_bits)
        Z = quantizer(quantizer((err + Z), num_bits=num_bits) * twiddles_quant,
            num_bits=num_bits)
        prev_out = quantizer(np.sum(np.real(Z[0])) / N, num_bits=num_bits)
    else:
        err = input_data[index] - prev_out
        Z = (err + Z) * twiddles_ideal
        prev_out = np.sum(np.real(Z[0])) / N

    out[index] = prev_out
```

Listing 2.3: R-SDFT DC component Python code part

2.4.2 Modulator-based SDFT considering the DC component

In Listing 2.4 is showed the algorithm of the M-SDFT considering the DC component in the signal reconstruction. All the rest of the function is equal to the M-SDFT function.

```
for index in range(input_size):
    if quant_en:
        err = quantizer(input_data[index] - prev_out, num_bits=num_bits)
        twiddle = quantizer(r_quant * np.exp(-1j * arg * index), num_bits=num_bits)
        prev_out = quantizer(np.sum(np.real(Z[0])) / N, num_bits=num_bits)
        Z = quantizer(quantizer(r_quant * Z, num_bits=num_bits) +
                quantizer(err * twiddle, num_bits=num_bits), num_bits=num_bits)
    else:
        err = input_data[index] - prev_out
        twiddle = r_ideal * np.exp(-1j * arg * index)
        prev_out = np.sum(np.real(Z[0])) / N
        Z = r_ideal * Z + err * twiddle

    out[index] = prev_out
```

Listing 2.4: M-SDFT DC component Python code part

Chapter 3

Results

In this chapter are presented the results from the Python code runs for different cases. The test bench is made with 200 waves with 3 variables listed below:

- amplitude: ten amplitudes normalized to 1 $(0.1, 0.2, ..., 1.0)$;
- frequency: ten frequencies in the range from the first on-bin frequency to 15kHz (are exluded DC and Nyquist components to simplify the comparison);
- on-bin and off-bin: useful to test the algorithm with frequencies equal to the bins in the spectrum and other frequencies slightly different to the bins of the spectrum.

Each wave is defined by

$$\begin{aligned} x(t) &= A\sin(2\pi f_{sw} t), \\ f_{sw} &= f_{bin}(1-d), \\ f_{bin} &= \frac{f_s}{N_0}. \end{aligned}$$

The N_0 time samples are distributed linearly over the interval from 0 to N_0/f_s. where A is the amplitude of the wave, f_s is the sampling frequency, N_0 is the number of wave samples and of frequency bins and d is the off-bin factor (set to 0.25 in this analysis).

Then are analyzed 3 different cases:

- ideal spectrum used for comparison with next steps;
- 32, 24, 16, 12 and 8 bits quantization cases;
- the previous point adding the consideration of the PCM error conductance.

To evaluate the behaviour of these algorithms under certain conditions are used the following metrics:

- magnitude error vs frequency

$$E_{mag} = \frac{\||X_{algorithm}| - |X_{FFT}|\|_2}{\||X_{FFT}|\|_2}$$

- complex (phase) error vs frequency

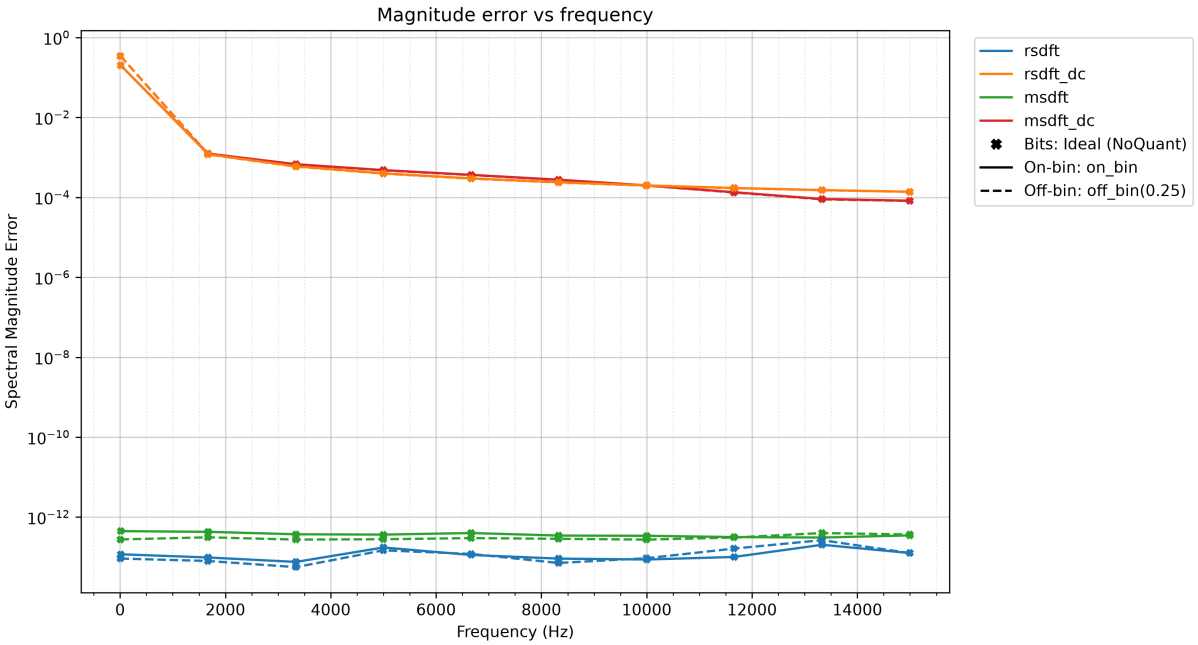


Figure 3.1: Ideal results case analysis, magnitude error vs frequency. $r = 1$ for every algorithm

$$E_{complex} = \frac{\|X_{algorithm} - X_{FFT}\|_2}{\|X_{FFT}\|_2}$$

- amplitude vs frequency vs complex (phase) error
- amplitude vs frequency vs magnitude error

At the beginning is considered the constant $r = 1$ then are showed the results with proper r for each algorithm.

3.1 Ideal (float64)

For this analysis has been crucial the precision given by numpy Python module [2] because of its optimized work. Is already seen its usage in Listing 2.1, Listing 2.2, Listing 2.3 and Listing 2.4.

3.1.1 Magnitude error vs frequency

From a magnitude error point of view M-SDFT and R-SDFT differs from the FFT by a little error, instead the approximations in the "DC" versions are too big to have an error so small. This error measures only the magnitude of the calculated elements so if in one application is not needed the phase (for example when is sent a costant wave) the DC versions could be implemented.
The magnitude-error results are shown in Fig. 3.1.

analysis ??? $r < 1$

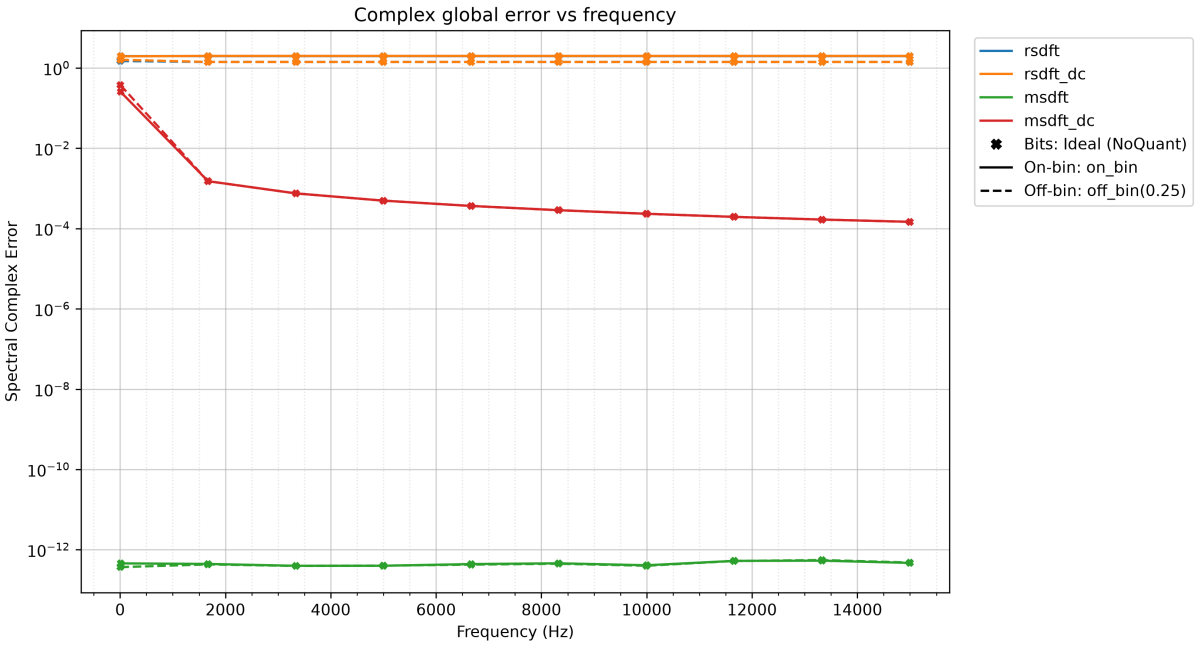


Figure 3.2: Ideal results case analysis, complex error vs frequency, on-bin case. $r = 1$ for every algorithm

3.1.2 Complex error (phase) vs frequency

In the Fig.3.2 are showed results of all algorithm analysis. The results confirm what is mentioned before about the stability of the algorithms, M-SDFT more stable than the R-SDFT due to the pole position. Even if the analysis is done using the $float64$ precision a small imperfection in the modulus of the twiddle factor > 1 creates a recursive error in the R-SDFT algorthm in fact is showed a huge error in comparison with the M-SDFT. About the "DC" algorithms the error was expected to be greater than the initial algorithms becasue of the reduced harmonic considerations.

analysis ??? $r < 1$

3.1.3 Heatmaps

In this section are commented some results related to the amplitude, the frequency and the error (magnitude and complex). In Fig.3.3 and in Fig.3.4 can be seen how the amplitude doesn't affects the comparison in the ideal case.

analysis ??? $r < 1$

3.2 Quantized versions

In this section is used a custom function exposed in Listing 3.1 for apply the quantization to the waves and the twiddle factors. The idea behind this function is to have the possiblity to use also non conventional number of bits (for example 5 bits instead of 8).

```
1  def quantizer(signal, num_bits:int=8, max_amplitude:float=1.0):
```

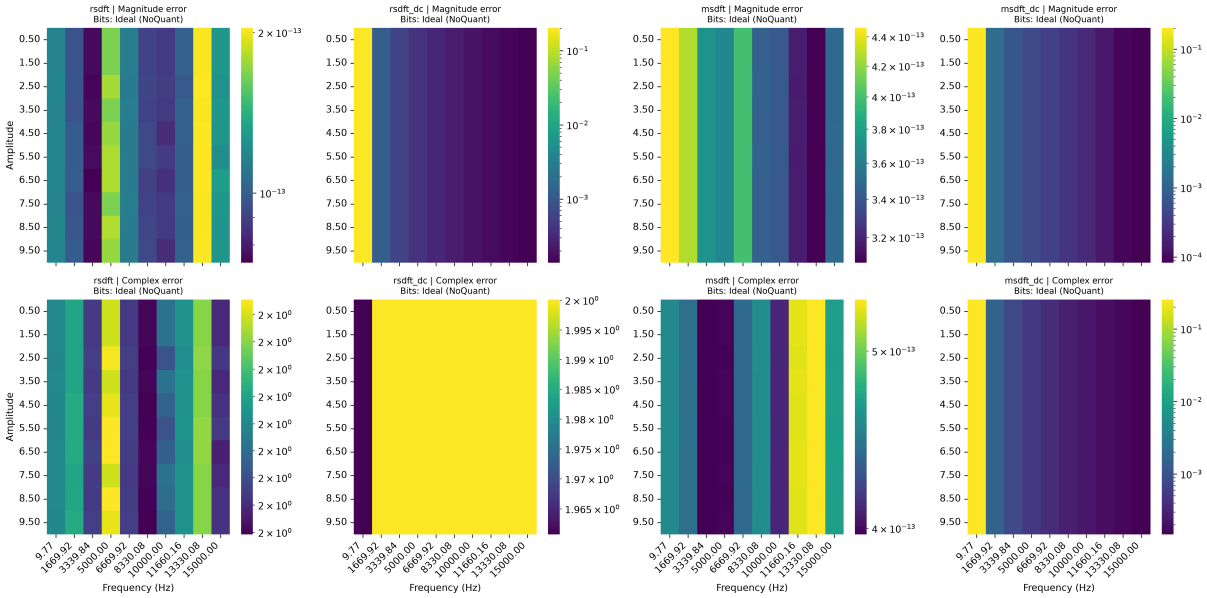


Figure 3.3: Heatmap on-bin (amplitude vs frequency vs errors)

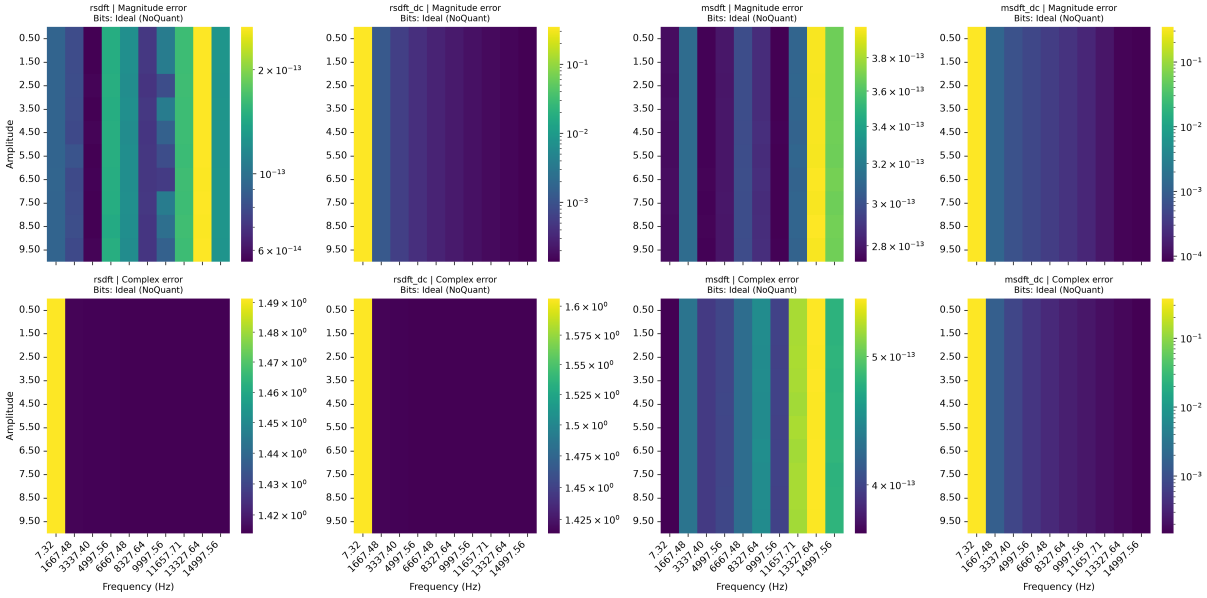


Figure 3.4: Heatmap off-bin (0.25) (amplitude vs frequency vs errors)

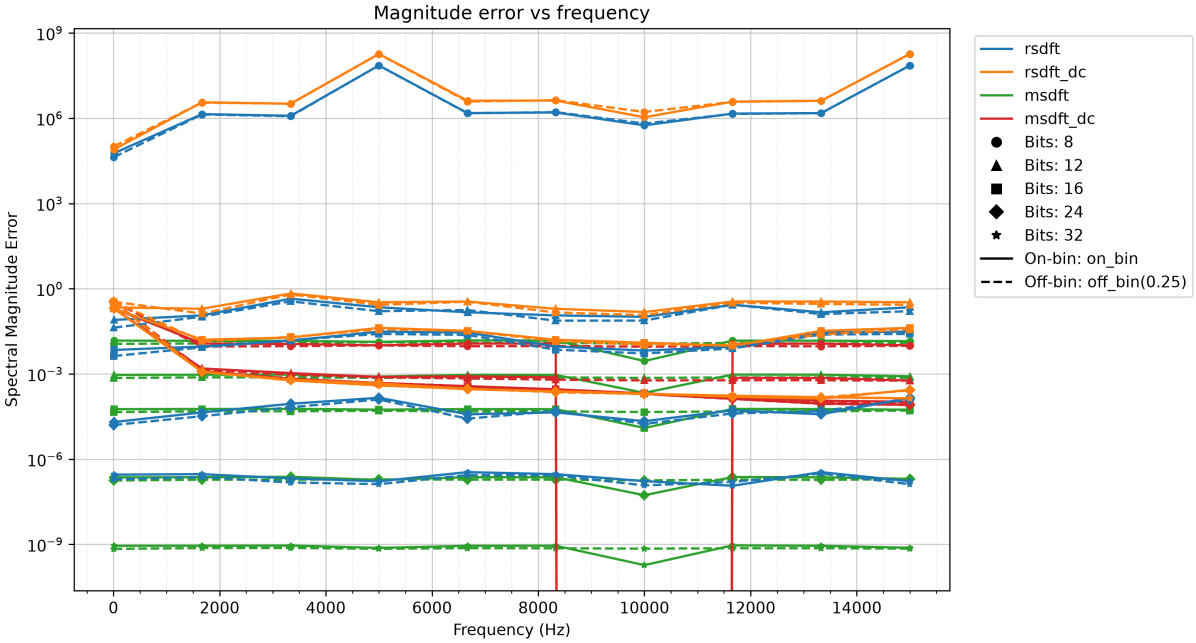


Figure 3.5: Magnitude error vs frequency for different quantization cases. $r = 1$ for every algorithm

```
    if num_bits == "no quant":
        raise Exception("Quantization is disabled. Please provide a valid number of
            bits.")

    def q(s):
        half_levels = 2 ** (num_bits - 1)
        max_bits = half_levels      # max 128 with 8 bits
        min_bits = -half_levels + 1 # min -127 with 8 bits

        num = max_amplitude * 2
        denum = half_levels * 2
        resolution = num / denum

        nums = np.round(s / resolution)
        # nums = np.floor(s / resolution)       # wrong idea
        return nums * resolution

    if np.iscomplexobj(signal):
        return q(np.real(signal)) + 1j * q(np.imag(signal))
    else:
        return q(signal)
```

Listing 3.1: Custom quantization funcion Python code

3.2.1 Magnitude error vs frequency

In Fig.3.5 are showed results of the magnitude error vs frequency for different quantization cases. What is clear is the R-SDFT algorithm is more sensitive to the quantization than the M-SDFT algorithm. There is also a freuency where seems the M-SDFT DC version collapses. This behaviour have to be investigated in the future.
In Fig.3.6 are showed results of the magnitude error vs number of bits for different quantization cases. As expected teh R-SDDFT algorithm presents an higher error than the M-SDFT algorithm.

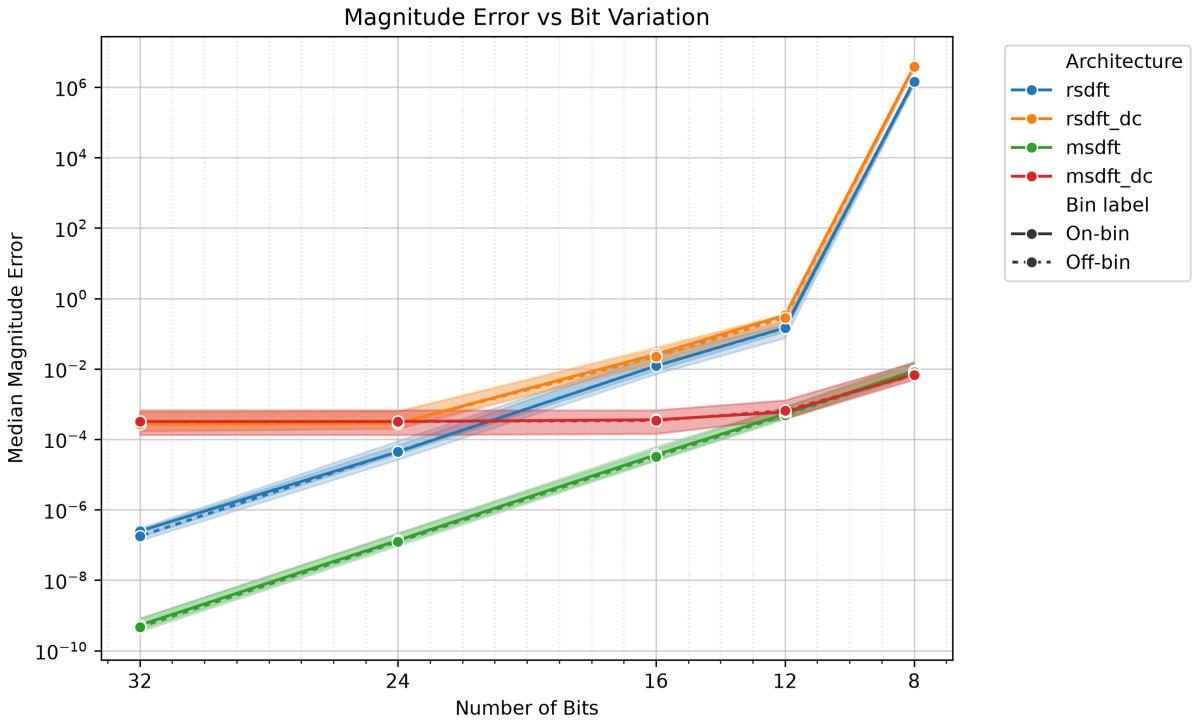


Figure 3.6: Magnitude error vs number of bits for different quantization cases. $r = 1$ for every algorithm

3.2.2 Complex error (phase) vs frequency

In Fig.3.7 are showed results of the complex error vs frequency for different quantization cases. In this figure the R-SDFT is barely visible but in above 10^0. Instead the M-SDFT is much more visible and clearly moves down with the incresing of number of bits.
It seems there is a frequency where even the M-SDFT collapses also with the DC version. This behaviour have to be investigated in the future.
In Fig.3.8 are showed results of the complex error vs number of bits for different quantization cases. What can be done is to compare these results with the previous magnitude error results and appreciate that the complex error is more sensitive to the quantization than the magnitude one.

3.2.3 Heatmaps

3.3 Circuital error

For the analysis of this section is used a function to extract random numbers to simulate the uncertainty of the PCM cell. From the Fig. 18 of [1] as been choosen 6% as uncertainty of the PCM single cell. For this reason are chosen

```
LOW_ERR = -0.06
HIGH_ERR = 0.06
```

Listing 3.2: PCM cell uncertainty parameters

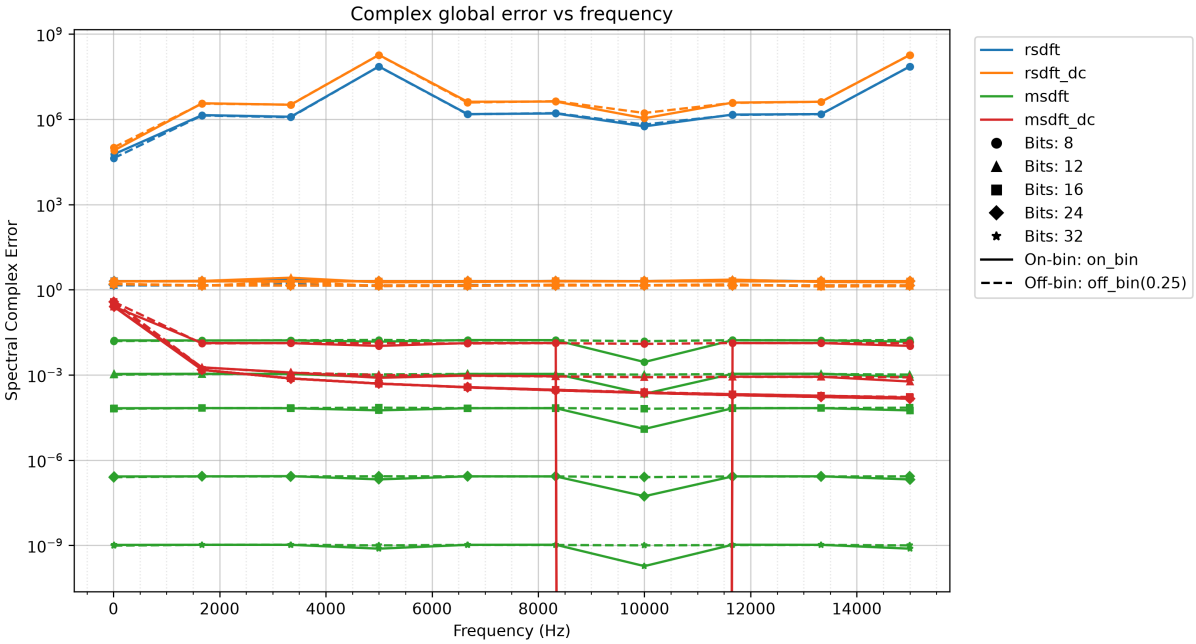


Figure 3.7: Complex error vs frequency for different quantization cases. $r = 1$ for every algorithm

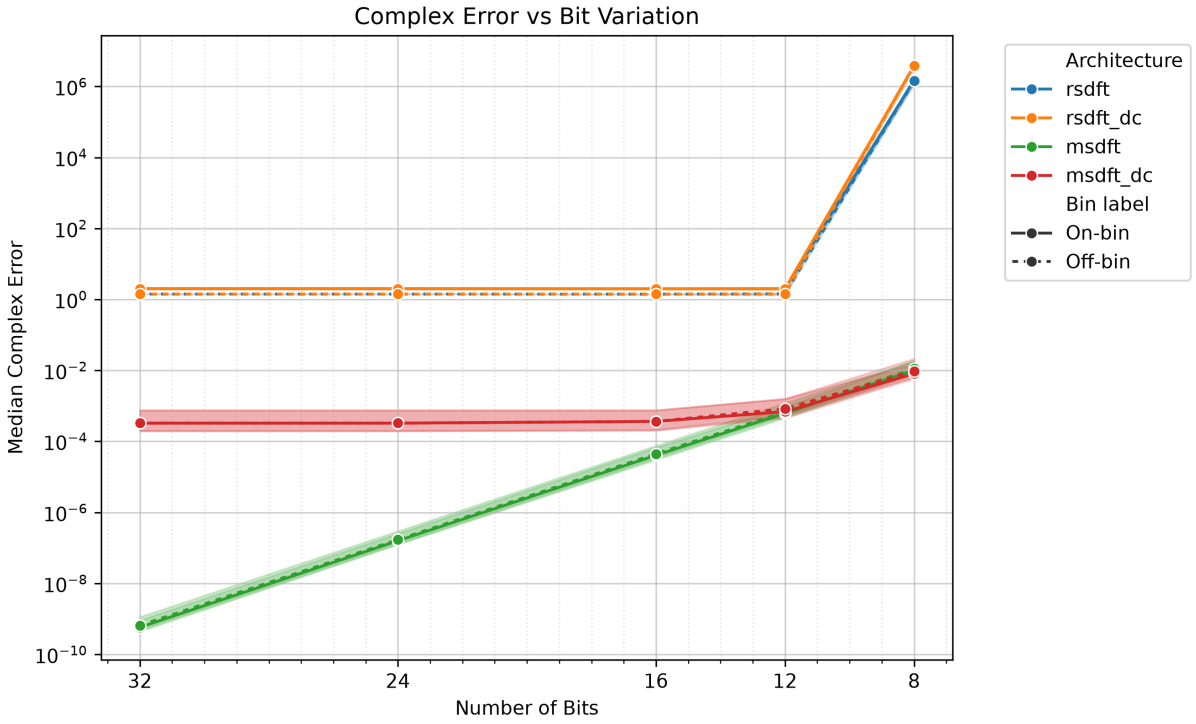


Figure 3.8: Complex error vs number of bits for different quantization cases. $r = 1$ for every algorithm

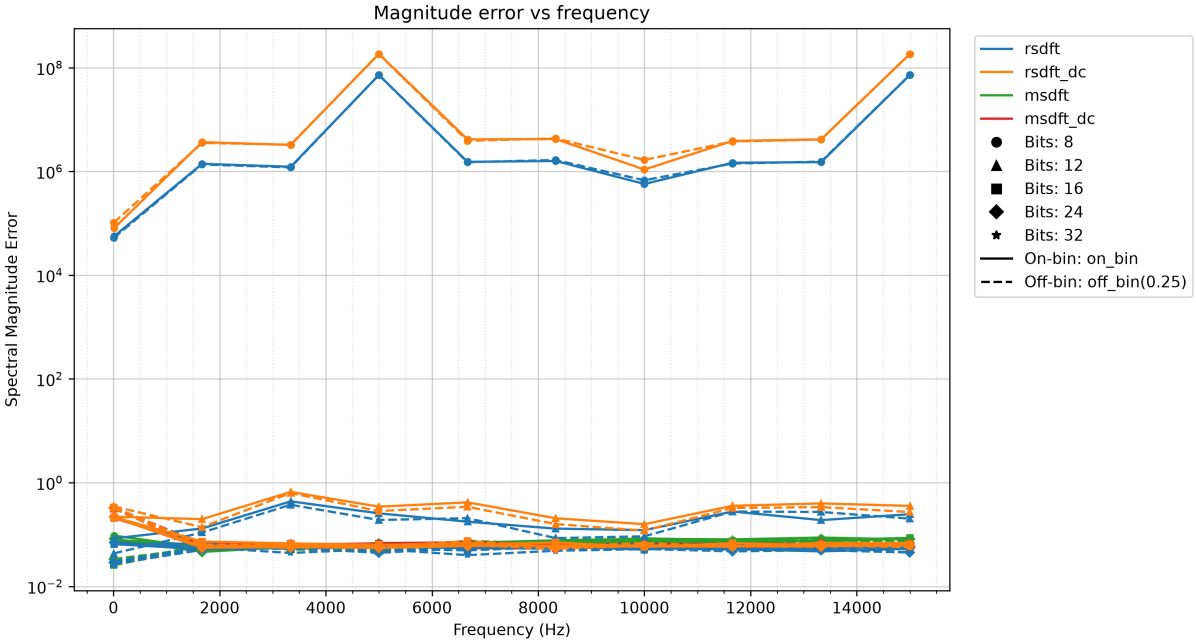


Figure 3.9: Magnitude error vs frequency for different quantization cases and with hardware error. $r = 1$ for every algorithm

and are used in the Listing 3.3 to apply the simulated uncertainty error.

```
k_vec = np.arange(N)
if hw_err:
    rng = np.random.default_rng()
    k_vec = k_vec + rng.uniform(low=LOW_ERR, high=HIGH_ERR, size=N)
arg = 2 * np.pi * k_vec / N
```

Listing 3.3: PCM cell introduced error

For the fact is supposed to work with normalized amplitude the Listing 3.2 defines 6% of 1.

3.3.1 Magnitude error vs frequency

In Fig.3.9 are showed results of the magnitude error vs frequency for different quantization cases. Like before the R-SDFT algorithm is more sensitive to the quantization than the M-SDFT algorithm. But what is interesting here is that the M-SDFT algorithm shows a more pronunced error at all frequencies. Instead the R-SDFT seems to be less affected by the hardware error since the error is like the previous case.
In Fig.3.10 are showed results of the magnitude error vs number of bits for different quantization cases.

3.3.2 Complex error (phase) vs frequency

TO COMPLETE

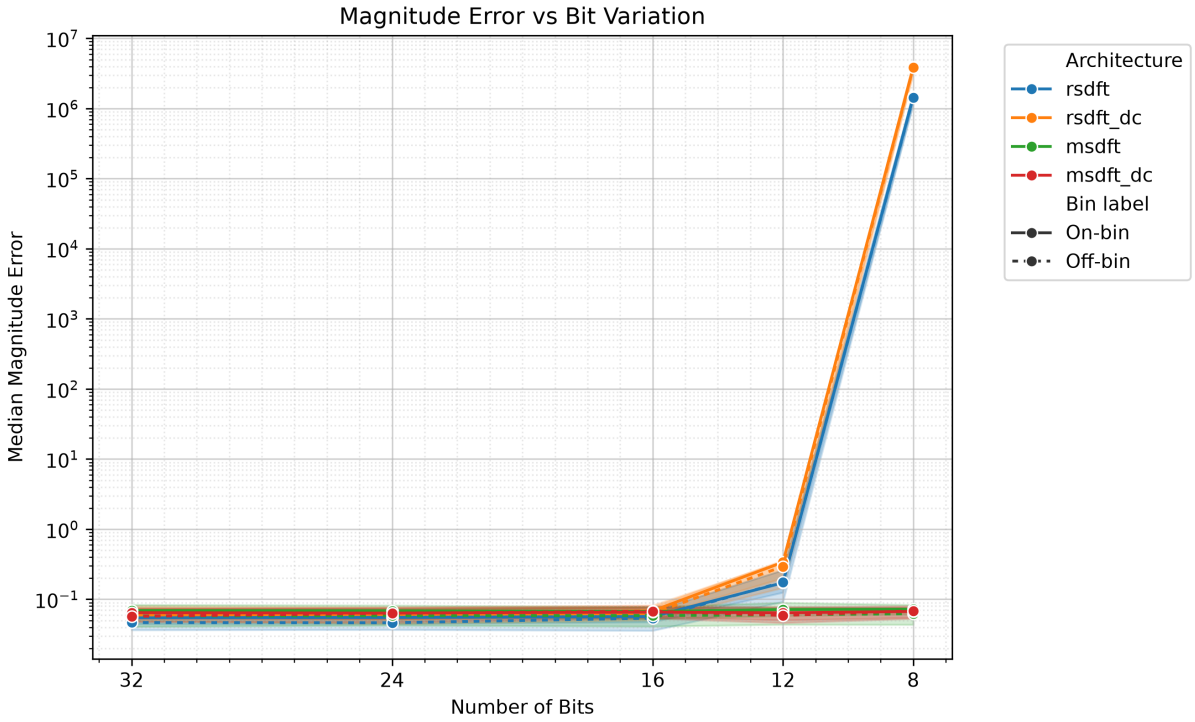


Figure 3.10: Magnitude error vs number of bits for different quantization cases and with hardware error. $r = 1$ for every algorithm

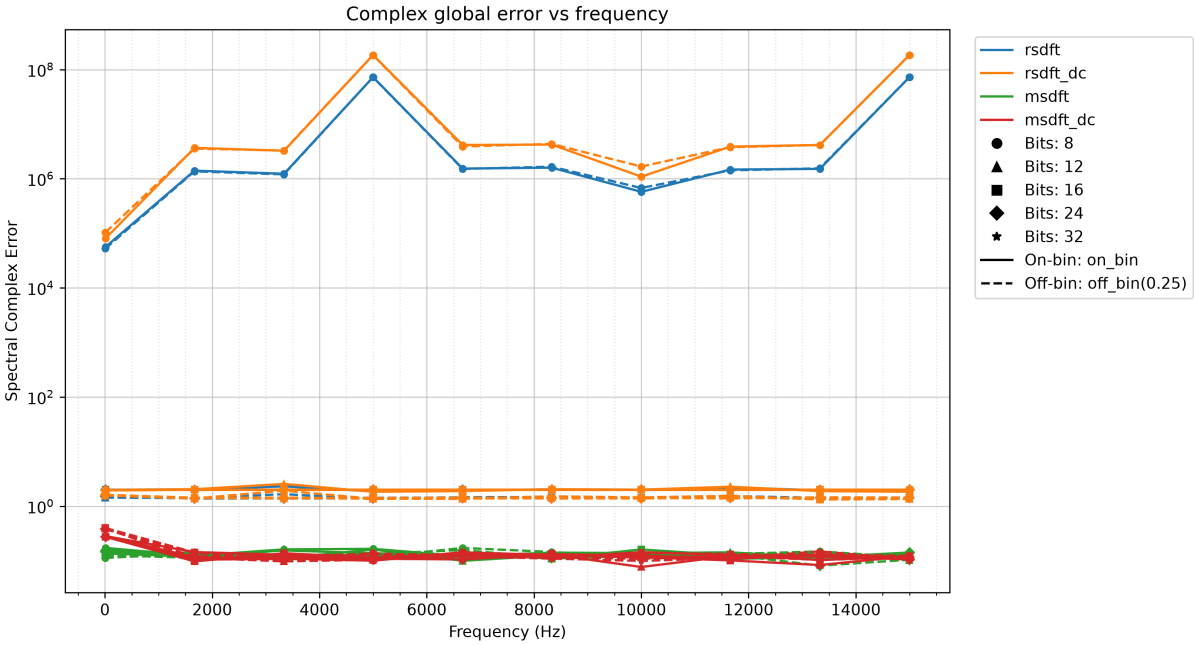


Figure 3.11: Complex error vs frequency for different quantization cases and with hardware error. $r = 1$ for every algorithm

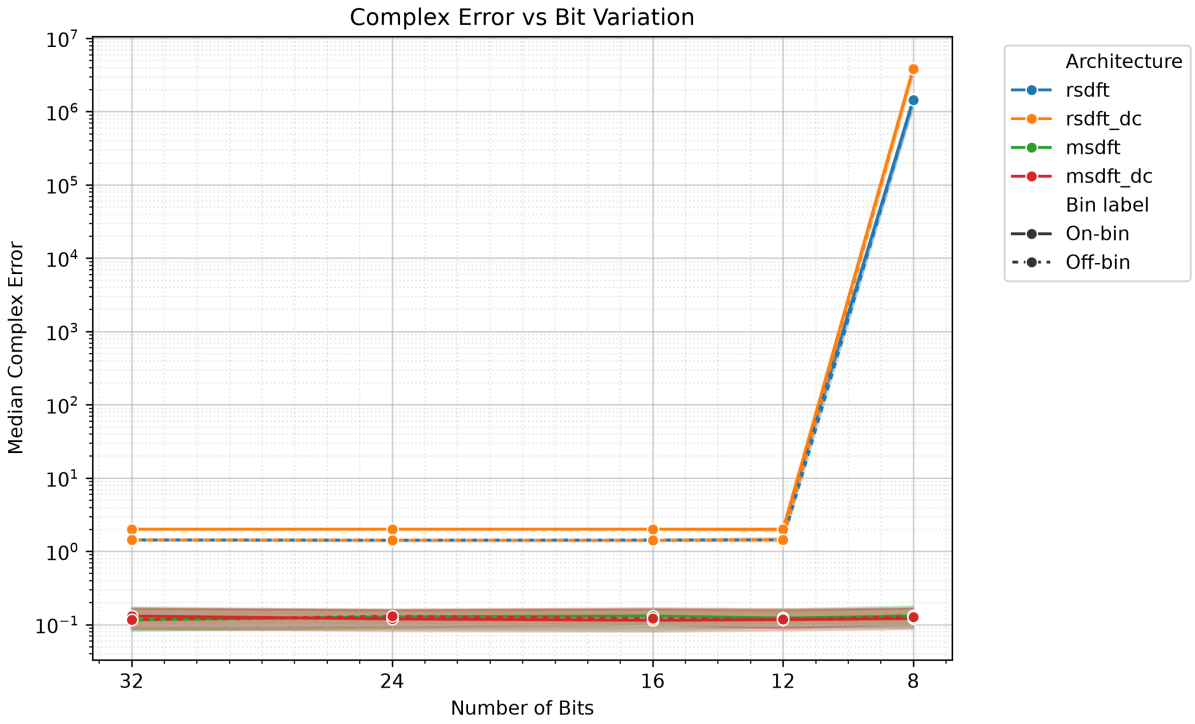


Figure 3.12: Complex error vs number of bits for different quantization cases and with hardware error. $r = 1$ for every algorithm

3.3.3 Heatmaps

Chapter 4

Electric circuit idea

In this section is explored the hardware implementation of a small part of the R-SDFT algorithm. As is showed in Fig.4.1 Is builded with a digital part where $x[n]$ and $y[n]$ are used to calculate the error $e[n] = x[n] - y[n]$. Each bit of the error is used to control a switch in series with a PCM resistor. When the switch is closed the resistor is connected to ground letting the current flow through it. Different resistors connected to the same node are used to create a weighted sum of the currents flowing through them. The total current is then summed with all other currents in the other branches each one referred to a different bin of the DFT. The total current is then converted into a digital quantity and divided by the number of bins (see Fig.2.3).

4.1 Simulations

In this section is presented the simulations of a small prototype of the circuit idea. In this implementation are used 3 bits PCM resistors. In Fig.4.2 is showed the simulated prototype. The input signal is an hypotetical error from the input-recostruction subtraction signals. The total current is then measured showing the second Kirkhhoff's law used in this MAC operation.

As is described in the R-SDFT section the time dependent formnula is built summing two elements: $e[n]$ and the previous spectrum coefficient $X_k[n-1]$ and both are multiplied by the same twiddle factor.

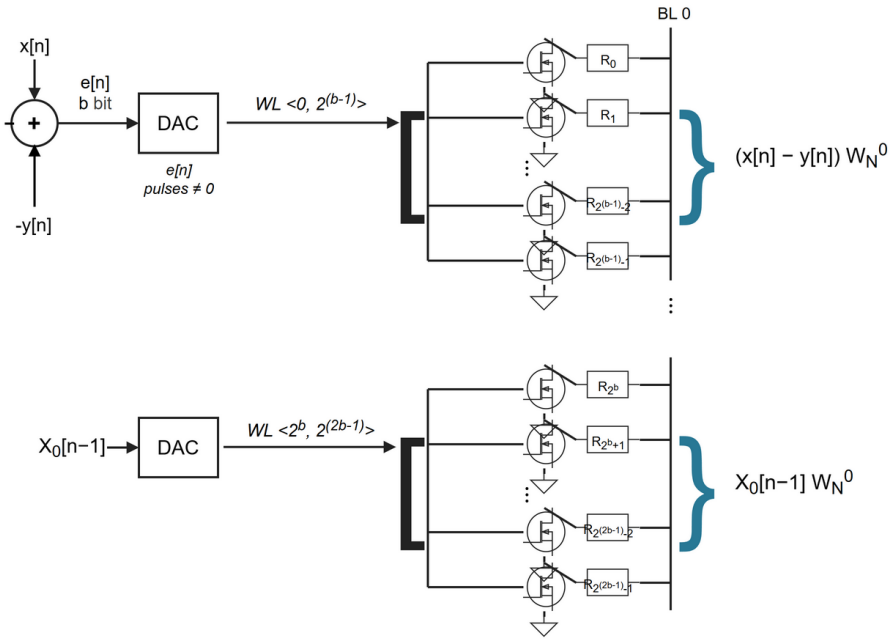


Figure 4.1: Circuit idea for one spectrum coefficient calculated by R-SDFT algorithm

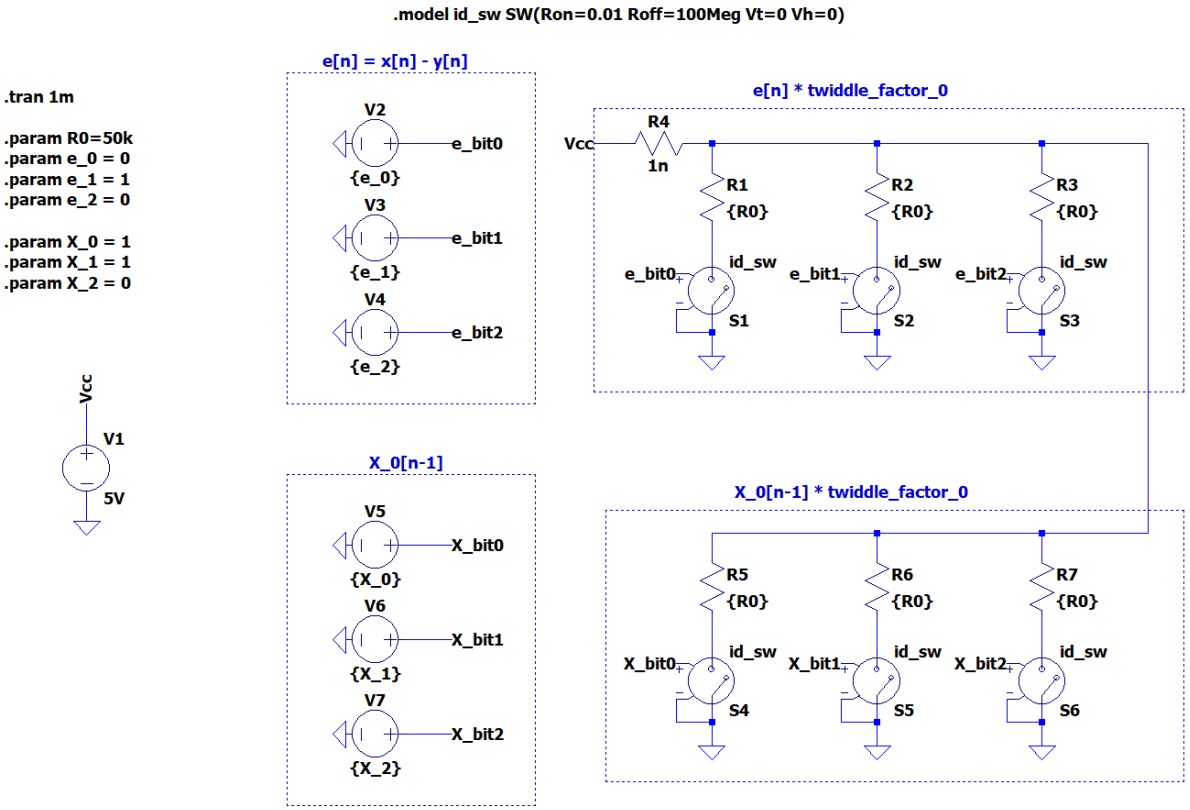


Figure 4.2: Circuit prototype implemented in LTspice. The circuit simulates a 3 bit structure

Chapter 5

Conclusions

Applications where the complex error is not important:
1. Hearingg aids, the human ear needs more the magnitude than the phase of a voice or a sound signal. TO CHECK
2. Constant waves, for example in a telegraph system or a 0/1 signal
3. Power measurement, the power is calculated with the magnitude of the signal.
Applications where the complex error is important:
1. Radar, the phase of the signal is important to understand the distance, the direction and the velocity of an object.
2. Signal reconstruction 3. Filter analyusis, the phase of the signal is important to understand the behavior of a filter.

Bibliography

[1] Alessio Antolini, Eleonora Franchi Scarselli, Antonio Gnudi, Marcella Carissimi, Marco Pasotti, Paolo Romele, and Roberto Canegallo. Characterization and programming algorithm of phase change memory cells for analog in-memory computing. *Materials*, 14(7):1624, 2021.

[2] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[3] MODIFY. Article title. *JOURNAL NAME*, 322(10):891–921, 1905.